1. IDENTITAS PRAKTIKUM

Nama	Riza Zaelani Dwi P
NIM	2507054
Kelas	Sistem informasi B
Mata kuliah	Praktikum Basis Data
Topik praktikum	Konfigurasi Index pada PostgreSQL
DBMS	PostgreSQL
Perangkat lunak	DBeaver dan DBngin

2. TUJUAN PRAKTIKUM

Praktikum ini bertujuan untuk memahami konsep dan fungsi indexing pada basis data, membuat database dan tabel menggunakan PostgreSQL, menyiapkan dataset sederhana, memverifikasi jumlah data, membuat single-column index dan composite index, serta memeriksa index yang terbentuk melalui katalog sistem PostgreSQL.

3. LINGKUNGAN PRAKTIKUM

Praktikum dilaksanakan menggunakan PostgreSQL pada lingkungan lokal DBngin dan dikelola melalui aplikasi DBeaver. Informasi teknis yang digunakan dicatat pada tabel berikut.

Komponen	Keterangan
Sistem operasi	.....
Versi PostgreSQL	Isi berdasarkan hasil SELECT version()
Spesifikasi komputer	.....
Nama database	db_praktikum

Perintah untuk memeriksa versi PostgreSQL:

```
SELECT version();
```

Hasil pemeriksaan versi PostgreSQL:

[Tempelkan screenshot di sini.]

4. DATASET

Dataset praktikum terdiri atas tiga tabel, yaitu products, customers, dan orders. Jumlah data yang ditargetkan sesuai modul adalah 40 produk, 30 pelanggan, dan 60 pesanan. Jumlah aktual dicatat berdasarkan hasil pemeriksaan di PostgreSQL.

Nama tabel	Jumlah target
products	40
customers	30
orders	60

Perintah untuk memeriksa jumlah data:

```
SELECT COUNT(*) FROM products;

SELECT COUNT(*) FROM customers;

SELECT COUNT(*) FROM orders;
```

Bukti hasil pemeriksaan dataset:

[Tempelkan screenshot di sini.]

5. IMPLEMENTASI PRAKTIKUM

5.1 Pembuatan database

Database yang digunakan dalam praktikum ini bernama db_praktikum. Database dibuat melalui koneksi PostgreSQL, kemudian koneksi DBeaver diarahkan ke database tersebut untuk menjalankan proses pembuatan tabel, pengisian dataset, dan konfigurasi index.

```
CREATE DATABASE db_praktikum;

SELECT current_database();
```

Bukti pembuatan dan pemilihan database:

[Tempelkan screenshot di sini.]

5.2 Pembuatan tabel

Tiga tabel digunakan untuk menyimpan data produk, pelanggan, dan pesanan. Tabel orders memiliki kolom customer_id sebagai foreign key yang merujuk ke customer_id pada tabel customers.

```
CREATE TABLE products (
    product_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    category VARCHAR(50) NOT NULL,
    price DECIMAL(12,2) NOT NULL,
    stock_quantity INT NOT NULL
);


CREATE TABLE customers (
    customer_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    city VARCHAR(50) NOT NULL
);


CREATE TABLE orders (
    order_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id INT NOT NULL,
    order_date DATE NOT NULL,
    total_amount DECIMAL(12,2) NOT NULL,
    CONSTRAINT fk_orders_customer FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
);
```

Bukti tabel berhasil dibuat:

[Tempelkan screenshot di sini.]

5.3 Pengisian dan verifikasi dataset

Data dimasukkan ke dalam masing-masing tabel sesuai dataset pada file SQL praktikum. Jumlah baris diperiksa menggunakan COUNT(*). Hasil aktual harus dituliskan berdasarkan output yang benar-benar tampil pada DBeaver.

```
SELECT COUNT(*) FROM products;
SELECT COUNT(*) FROM customers;
SELECT COUNT(*) FROM orders;
```

Bukti hasil jumlah data:

[Tempelkan screenshot di sini.]

5.4 Pembuatan single-column index

Single-column index merupakan index yang dibuat pada satu kolom. Pada praktikum ini, index digunakan untuk kolom yang relevan dengan pencarian, yaitu category dan price pada tabel products, order_date pada tabel orders, serta city pada tabel customers.

```
CREATE INDEX IF NOT EXISTS idx_products_category ON products (category);
CREATE INDEX IF NOT EXISTS idx_products_price ON products (price);
```

```
CREATE INDEX IF NOT EXISTS idx_orders_date ON orders (order_date);

CREATE INDEX IF NOT EXISTS idx_customers_city ON customers (city);
```

Tabel ringkasan single-column index:

Nama index	Tabel/kolom	Hasil
idx_products_category	products(category)	Isi setelah verifikasi
idx_products_price	products(price)	Isi setelah verifikasi
idx_orders_date	orders(order_date)	Isi setelah verifikasi
idx_customers_city	customers(city)	Isi setelah verifikasi

Bukti pembuatan single-column index:

[Tempelkan screenshot di sini.]

5.5 Pembuatan composite index

Composite index merupakan index yang mencakup dua kolom atau lebih. Index pada tabel orders dibuat dengan kombinasi customer_id dan order_date agar sesuai dengan pola pencarian yang menggunakan kedua kolom tersebut.

```
CREATE INDEX IF NOT EXISTS idx_orders_customer_date

ON orders (customer_id, order_date);
```

Bukti pembuatan composite index:

[Tempelkan screenshot di sini.]

5.6 Pemeriksaan index

Index yang telah dibuat diperiksa menggunakan katalog pg_indexes. Pemeriksaan dilakukan untuk memastikan nama index, tabel, dan definisi kolom yang diindeks.

```
SELECT indexname, indexdef FROM pg_indexes WHERE schemaname = 'public' AND
tablename = 'products';

SELECT indexname, indexdef FROM pg_indexes WHERE schemaname = 'public' AND
tablename = 'customers';

SELECT indexname, indexdef FROM pg_indexes WHERE schemaname = 'public' AND
tablename = 'orders';
```

Bukti hasil pemeriksaan pg_indexes:

[Tempelkan screenshot di sini.]

6. JAWABAN PERTANYAAN DISKUSI

1. Apa fungsi index pada basis data?

Index membantu PostgreSQL menemukan baris yang memenuhi kondisi pencarian dengan lebih efisien. Index dapat mendukung pencarian, penyaringan, pengurutan, dan join, bergantung pada bentuk query dan rencana eksekusi.

2. Apakah setiap kolom harus diberikan index?

Tidak. Index sebaiknya dibuat pada kolom yang sering digunakan dalam kondisi WHERE, JOIN, atau pengurutan. Pembuatan index perlu mempertimbangkan kebutuhan query dan biaya pemeliharaannya.

3. Mengapa index dapat menambah biaya INSERT, UPDATE, dan DELETE?

Perubahan data dapat mengharuskan PostgreSQL memperbarui index yang terkait. Karena itu, semakin banyak index yang harus dipelihara, semakin besar potensi biaya operasi perubahan data.

4. Apa perbedaan single-column index dan composite index?

Single-column index dibuat pada satu kolom, sedangkan composite index dibuat pada dua kolom atau lebih.

5. Mengapa urutan kolom pada composite index penting?

Urutan kolom memengaruhi jenis query yang dapat memanfaatkan index. Index (customer_id, order_date) terutama sesuai untuk query yang menggunakan customer_id dan dapat membantu query berdasarkan customer_id saja; hasilnya tidak sama dengan index berurutan (order_date, customer_id).

6. Bagaimana index membantu operasi JOIN?

Index pada kolom yang digunakan untuk menghubungkan tabel dapat membantu PostgreSQL menemukan baris terkait dengan lebih efisien, bergantung pada jumlah data dan rencana eksekusi.

7. Apa risiko membuat terlalu banyak index?

Terlalu banyak index menggunakan ruang penyimpanan dan dapat menambah biaya INSERT, UPDATE, dan DELETE karena index perlu dipelihara. Index harus dipilih berdasarkan kebutuhan query.

7. REFLEKSI PRAKTIKUM

1. Index mana saja yang berhasil dibuat, dan pada kolom apa?

Isi berdasarkan hasil pemeriksaan pg_indexes. Cantumkan nama index dan kolom yang benar-benar muncul.

2. Apa konsekuensi jika sebuah tabel memiliki terlalu banyak index?

Terlalu banyak index dapat menambah kebutuhan penyimpanan dan memperlambat operasi perubahan data karena index terkait harus dipelihara.

3. Bagaimana menentukan apakah sebuah kolom layak diberikan index?

Pertimbangkan pola query yang sering dijalankan, kolom yang digunakan dalam WHERE/JOIN/ORDER BY, selektivitas kolom, serta biaya penyimpanan dan pemeliharaan index.

8. KESIMPULAN

Praktikum konfigurasi index pada PostgreSQL menunjukkan bahwa index perlu dipilih berdasarkan pola query dan kolom yang digunakan. Single-column index diterapkan pada kolom tertentu, sedangkan composite index menggabungkan lebih dari satu kolom untuk mendukung pola pencarian yang sesuai. Keberadaan index diverifikasi melalui katalog pg_indexes. Kesimpulan akhir perlu disesuaikan dengan index yang benar-benar berhasil dibuat dan hasil pemeriksaan dataset di DBeaver.